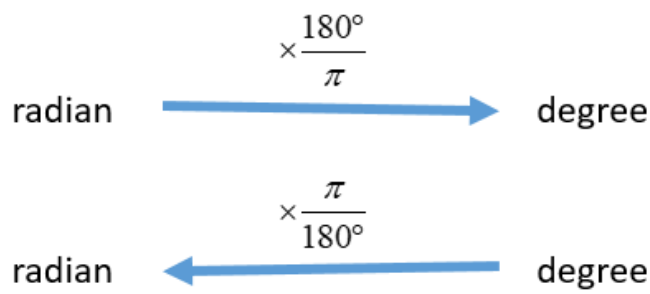
What is Inverse Kinematics?

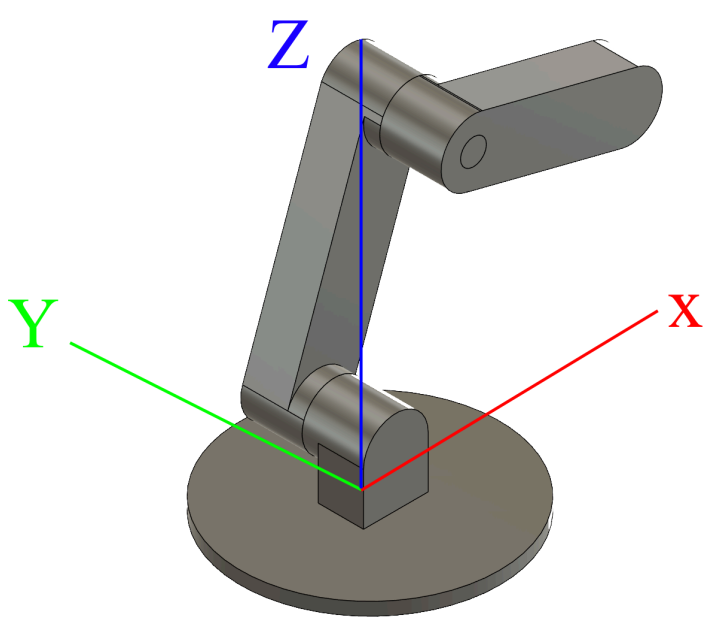
In the world of servos and motors they do not speak motion in the language of angles, instead they understand the PWM (pulse width modulation) Language. For eg you want a servo to rotate 90°, to perform a rotation that is similar to 90° we'll need to send a pulse (a square wave) of width 1600 or similar. This all can be calculated manually, but things will start to pile up once we have a 7 DOF arm moving in 3d space, hence by using geometry, algebra and bit of basic trigonometry we calculate the rotation required for each joint of an to reach and defined (x,y,z) location in a 3d space.

Solving the angles

In our example pi.FO arm we have a basic 2DOF frame, the baselink is fixed in 3d space and the shoulder and the arm joints perform constrained motion

Generally Radians are used for calculations but that can be easily converted to degrees, which is more familiar to everyone. So we will use degree for the units.

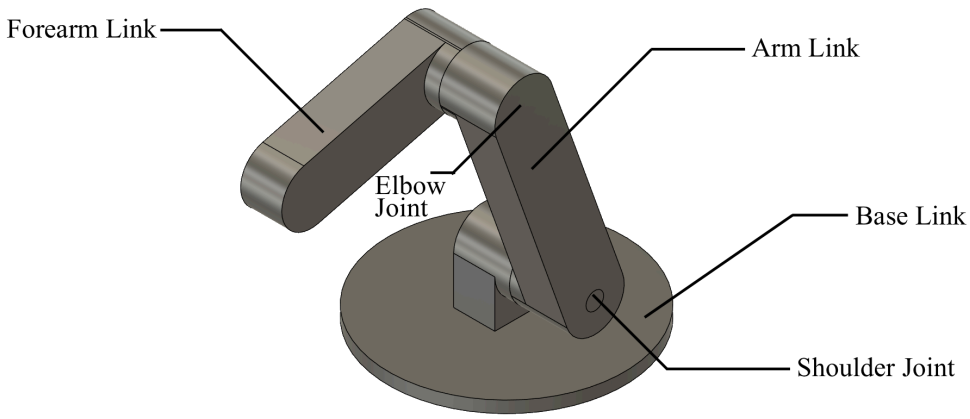




We will be using the right hand coordinate system for all the calculations

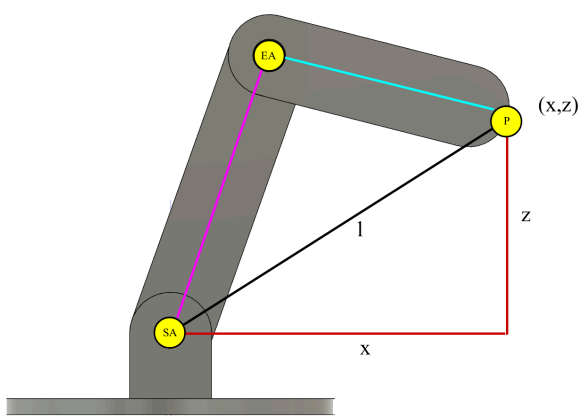
OVERVIEW:

To move the arm tip (P) to a specific target coordinate (x, z), we treat the arm structure as a triangle. We use two physical constants from the CAD model:



STEP 1: CALCULATE THE "HYPOTENUSE" (L)

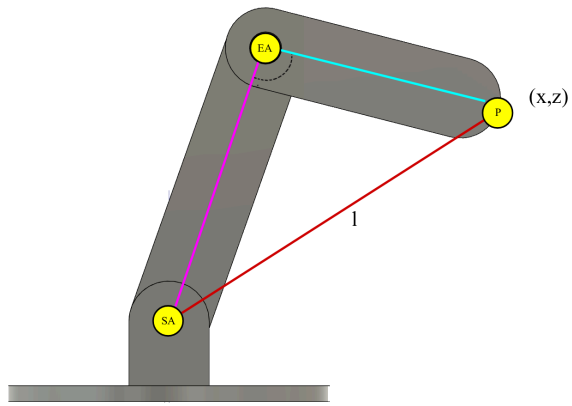
First, we find the direct distance from the base motor to our target coordinates using Pythagoras theorem. Think of this as an imaginary line (L) connecting the shoulder to the tip.



$$L = \sqrt{x^2 + z^2}$$

STEP 2: FINDING THE ELBOW ANGLE

(EA) Now we have a triangle where all three sides are known: FL, AL, and the calculated line l. We use the Law of Cosines to find the angle at the Elbow Joint. This determines how much the Arm Link folds relative to the Forearm Link.

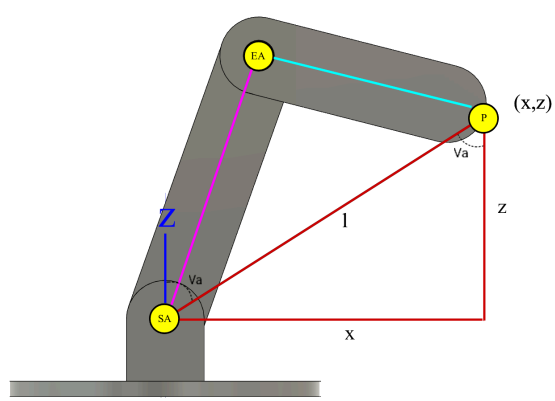


$$\cos(EA) = (FL^2 + AL^2 - L^2) / (2 * FL * AL)$$

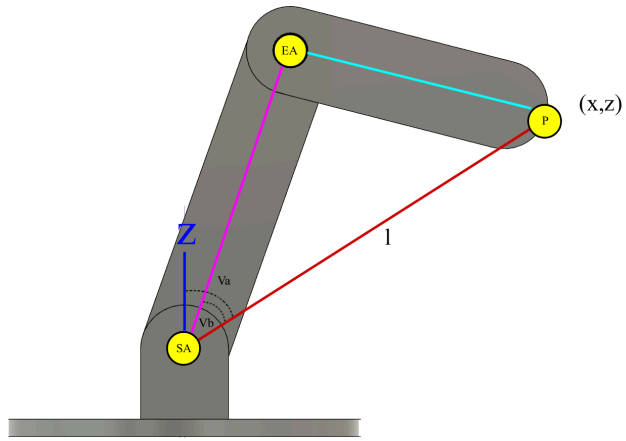
STEP 3: FINDING THE SHOULDER ANGLE (SA)

The Shoulder Joint needs to rotate the entire assembly to the correct height. This is calculated in two parts:

1. va: The angle of the target point (P) relative to the X-axis (calculated using atan2(z, x)).



2. vb: The internal angle between the Forearm Link (FL) and the target line l (calculated using the Law of Cosines).



The final Shoulder Angle (SA) is found by Subtracting these

$$SA = Va - Vb$$

STEP 4: THE "GAP" CORRECTION

In real-world hardware, the physical "zero" of your servo might not perfectly align with the mathematical "zero" shown in CAD. We apply a Gap Angle (GA) to calibrate the final output.

$$Final_Servo_PWM = Calculated_Angle - GA$$

FABRIK (Forward And Backward Reaching Inverse Kinematics)

A simple 2DOF upto 3DOF can be easily implemented via this approach, but as the DOF increases the calculations becomes heavier and slows down the script, hence for higher degree of freedom we will adapt FABRIK approach

What is FABRIK?

FABRIK is a kinematics algorithm, it calculates the angles by an iterative process. First it grabs the target point and then pulls the limb towards the origin. Then it starts from the origin and tries to reach the target point, this process iterates until the limb has reached close enough to reach the target point. For a supposed **7DOF arm** similar to human arm it would require a version of the FABRIK algorithm implemented in the lua script.